

SQL Übungsblatt JDBC und JPA

Alle Aufgaben beziehen sich auf die Chinook-Datenbank.

Aufgaben

1. Geben Sie die den Inhalt der Tabelle customer (Attribute customer_id und last_name) aus, indem sie mit Java auf die Tabelle im Schema „chinook“ zugreifen.
2. a) Dateneingabe: Erzeugen sie in java eine SQL Anweisung, welche genre_id 77 mit dem Namen “Pop 70er“ in die Tabelle genre hinzufügt

b) Datenänderung: Erzeugen sie in java eine SQL Anweisung, welche für die genre_id 77 den Namen auf „Pop Siebziger“ ändert
3. Referentielle Integrität und Fehlerbehandlung

a) Versuchen Sie einen Datensatz in die Tabelle employee einzufügen mit der emp_pro_id 77. (employee_id = 77, last_name = ‘Napf’, first_name = ‘Karl’, reports_to = 555)

b) Fügen sie einen try ... catch Block ein, um Fehler zu behandeln

c) Fügen Sie einen Datensatz in die Tabelle employee ein mit der emp_pro_id 555. (employee_id = 555, last_name = ‘Bär’, first_name = ‘Hans’). Versuchen Sie erneut einen Datensatz in die Tabelle employee einzufügen mit der emp_pro_id 77. (employee_id = 77, last_name = ‘Napf’, first_name = ‘Karl’, reports_to = 555) Sie können einen try... catch Block verwenden, müssen dies aber nicht
4. a) Schreiben Sie ein java-Programm, welches den Vornamen eines Mitarbeiters anhand des Nachnamens ermitteln kann. (Tabelle employee).
 - Der Nachname ist eine definierte Variable im Code des java Programms. (String lastname = "King";)
 - Fügen sie diesen String mit einer SQL-Abfrage "SELECT * FROM employee WHERE last_name = ". zusammen (wie in der Vorlesung).

- Mit der executeQuery-Methode führen sie diesen zusammengeführten SELECT-Befehl aus

b) Belegen Sie anschließend die Variable lastname mit einem Ausdruck, der immer wahr ergibt (Bsp siehe Vorlesung). Führen sie dann die executeQuery-Methode erneut aus. Es sollten nun alle Vornamen der Tabelle employee ausgegeben werden

=> potentiell Risiko für SQL Injection (falls die Eingabe des lastname über eine UI erfolgen würde)

c) Schreiben Sie ein zweites java-Programm mit der gleichen Funktionalität, welche ein PreparedStatement nutzt und damit SQL Injection verhindert.

5. a) Der Mitarbeiter mit der employee_id 2 scheidet aus dem Unternehmen aus. Schreiben ein sie java Programm und setzen sie für diesen Mitarbeiter den "last_name" auf 'inactive' in der Tabelle employee
- b) Ein neuer Mitarbeiter wird eingestellt. Fügen sie mit einem java Programm einen neuen Datensatz in die Tabelle employee ein. (employee_id = 192, last_name = 'Mayer', first_name = 'Patrick')
- c) Geben sie mit einem java Programm für alle Mitarbeiter in der Tabelle employee folgende Attribute aus: employee_id, last_name, first_name

Teil 2: Lösungen

Bitte beachten: Die Liste der Imports ist um die Übersichtlichkeit zu verbessern in den unten angefügten Lösungen nicht explizit aufgeführt:

```
import java.util.Properties;
import java.sql.DriverManager;
import java.sql.Connection;
import java.sql.Statement;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.DatabaseMetaData;
import java.sql.SQLException;
```

(eigentlich werden nicht alle benötigt)

Bitte jeweils am Anfang des java-Codes einfügen

Aufgabe 1.

```
public class Main {
    public static void main(String[] args) throws SQLException {
        String adresse =
"jdbc:postgresql://localhost:5432/postgres?currentSchema=%s";
        String schema = "chinook";
        String url = String.format(adresse, schema);
        Properties props = new Properties();
        props.setProperty("user", "postgres");
        props.setProperty("password", "postgres");

        // Verbindungsaufbau
        Connection conn = DriverManager.getConnection(url,
props);

        // SQL statement vorbereiten
        Statement st = conn.createStatement();
        String stcommand = "select * from chinook.customer;";

        // SQL Befehl ausführen
        ResultSet rs = st.executeQuery(stcommand);

        // Auswertung - Ausgabe der Tabelle Customer
        while (rs.next()) {
            Integer ID = rs.getInt("customer_id");
            String Name = rs.getString("last_name");
            System.out.print("      ");
            System.out.print(ID);
            System.out.print("      ");
            System.out.println(Name);
        }
    }
}
```

```
    }

    //  Freigabe der Ressourcen
        conn.close();
        st.close();
        rs.close();

    }

}
```

Aufgabe 2 a)

```
public class Main {
    public static void main(String[] args) throws SQLException {
        String adresse =
"jdbc:postgresql://localhost:5432/postgres?currentSchema=%s";
        String schema = "chinook";
        String url = String.format(adresse, schema);
        Properties props = new Properties();
        props.setProperty("user", "postgres");
        props.setProperty("password", "postgres");

        // Verbindungsaufbau
        Connection conn = DriverManager.getConnection(url,
props);

        // SQL statement vorbereiten
        Statement st = conn.createStatement();
        // Aufgabe Insert Befehl
        String stcommand = "INSERT INTO genre(genre_id,name)
VALUES (77,'Pop 70er');";

        // SQL Befehl ausführen
        st.executeUpdate(stcommand);

        //////////////////////////////////////
        // Tabelleninhalt ausgeben um Ergebnis anzuzeigen
        //////////////////////////////////////

        // SQL statement vorbereiten
        stcommand = "select * from genre;";

        // SQL Befehl ausführen
        ResultSet rs = st.executeQuery(stcommand);

        // Auswertung - Ausgabe der Tabelle Genre
        while (rs.next()) {
            Integer ID = rs.getInt("genre_id");
            String Name = rs.getString("name");
            System.out.print("      ");
            System.out.print(ID);
            System.out.print("      ");
            System.out.println(Name);
        }
    }
}
```

```

    }

    // Freigabe der Ressourcen
    conn.close();
    st.close();
    rs.close();

}

}

```

Aufgabe 2 b)

```

public class Main {
    public static void main(String[] args) throws SQLException {
        String adresse =
"jdbc:postgresql://localhost:5432/postgres?currentSchema=%s";
        String schema = "chinook";
        String url = String.format(adresse, schema);
        Properties props = new Properties();
        props.setProperty("user", "postgres");
        props.setProperty("password", "postgres");

        // Verbindungsaufbau
        Connection conn = DriverManager.getConnection(url,
props);

        // SQL statement vorbereiten
        Statement st = conn.createStatement();
        String stcommand = "UPDATE genre SET name = 'Pop
Siebziger' WHERE genre_id = 77;";

        // SQL Befehl ausführen
        st.executeUpdate(stcommand);

        //////////////////////////////////////
        // Tabelleninhalt ausgeben um Ergebnis anzuzeigen
        //////////////////////////////////////
    }
}

```

```

// SQL statement vorbereiten
    stcommand = "select * from genre;";

// SQL Befehl ausführen
    ResultSet rs = st.executeQuery(stcommand);

// Auswertung - Ausgabe der Tabelle Genre
    while (rs.next()) {
        Integer ID = rs.getInt("genre_id");
        String Name = rs.getString("name");
        System.out.print("      ");
        System.out.print(ID);
        System.out.print("      ");
        System.out.println(Name);
    }

// Freigabe der Ressourcen
    conn.close();
    st.close();
    rs.close();

}

```

Aufgabe 3 a)

```
public class Main {
    public static void main(String[] args) throws SQLException {
        String adresse =
"jdbc:postgresql://localhost:5432/postgres?currentSchema=%s";
        String schema = "chinook";
        String url = String.format(adresse, schema);
        Properties props = new Properties();
        props.setProperty("user", "postgres");
        props.setProperty("password", "postgres");

        // Verbindungsaufbau
        Connection conn = DriverManager.getConnection(url,
props);

        // SQL statement vorbereiten
        Statement st = conn.createStatement();

        // a) Insert Befehl
        String stcommand = "INSERT INTO employee (employee_id,
last_name, first_name, reports_to) VALUES (77,'Napf', 'Karl',
555);";
        //
        // a) SQL Befehl ausführen
        st.executeUpdate(stcommand);

        //////////////////////////////////////
        // Tabelleninhalt ausgeben um Ergebnis anzuzeigen
        //////////////////////////////////////

        // SQL statement vorbereiten
        stcommand = "select * from employee;";

        // SQL Befehl ausführen
        ResultSet rs = st.executeQuery(stcommand);

        // Auswertung - Ausgabe der Tabelle employee
        while (rs.next()) {
            Integer ID = rs.getInt("employee_id");
            String NName = rs.getString("last_name");
            String VName = rs.getString("first_name");
            System.out.print("      ");
        }
    }
}
```

```

        System.out.print(ID);
        System.out.print("      ");
        System.out.println(NName);
        System.out.println(VName);
    }

    // Freigabe der Ressourcen
    conn.close();
    st.close();
    rs.close();

}

}

```

Aufgabe 3 b)

```

public class Main {
    public static void main(String[] args) throws SQLException {
        String adresse =
"jdbc:postgresql://localhost:5432/postgres?currentSchema=%s";
        String schema = "chinook";
        String url = String.format(adresse, schema);
        Properties props = new Properties();
        props.setProperty("user", "postgres");
        props.setProperty("password", "postgres");

        // Verbindungsaufbau
        Connection conn = DriverManager.getConnection(url,
props);

        // SQL statement vorbereiten
        Statement st = conn.createStatement();

        // a) und b) Insert Befehl
        String stcommand = "INSERT INTO employee (employee_id,
last_name, first_name, reports_to) VALUES (77,'Napf', 'Karl',
555)";
        //
        // b) mit TRY CATCH
        try {
            st.executeUpdate(stcommand);
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}

```

```

};

////////////////////////////////////////
// Tabelleninhalt ausgeben um Ergebnis anzuzeigen
////////////////////////////////////////

// SQL statement vorbereiten
    stcommand = "select * from employee;";

// SQL Befehl ausführen
    ResultSet rs = st.executeQuery(stcommand);

// Auswertung - Ausgabe der Tabelle employee
    while (rs.next()) {
        Integer ID = rs.getInt("employee_id");
        String NName = rs.getString("last_name");
        String VName = rs.getString("first_name");
        System.out.print("      ");
        System.out.print(ID);
        System.out.print("      ");
        System.out.println(NName);
        System.out.println(VName);
    }

// Freigabe der Ressourcen
    conn.close();
    st.close();
    rs.close();

}

}

```

Aufgabe 3 c)

```

public class Main {
    public static void main(String[] args) throws SQLException {
        String adresse =
"jdbc:postgresql://localhost:5432/postgres?currentSchema=%s";
        String schema = "chinook";
        String url = String.format(adresse, schema);
        Properties props = new Properties();

```

```

        props.setProperty("user", "postgres");
        props.setProperty("password", "postgres");

// Verbindungsaufbau
        Connection conn = DriverManager.getConnection(url,
props);

// SQL statement vorbereiten
        Statement st = conn.createStatement();

// c) Insert Befehl
        String stcommand = "INSERT INTO employee (employee_id,
last_name, first_name) VALUES (555,'Napf', 'Karl');";
// c) SQL Befehl ausführen
        st.executeUpdate(stcommand);
// c) Insert Befehl
        stcommand = "INSERT INTO employee (employee_id,
last_name, first_name, reports_to) VALUES (77,'Napf', 'Karl',
555);";
// c) SQL Befehl ausführen
        st.executeUpdate(stcommand);

////////////////////////////////////
// Tabelleninhalt ausgeben um Ergebnis anzuzeigen
////////////////////////////////////

// SQL statement vorbereiten
        stcommand = "select * from employee;";

// SQL Befehl ausführen
        ResultSet rs = st.executeQuery(stcommand);

// Auswertung - Ausgabe der Tabelle employee
        while (rs.next()) {
            Integer ID = rs.getInt("employee_id");
            String NName = rs.getString("last_name");
            String VName = rs.getString("first_name");
            System.out.print("      ");
            System.out.print(ID);
            System.out.print("      ");
            System.out.println(NName);
            System.out.println(VName);
        }

```

```
    }

    //  Freigabe der Ressourcen
        conn.close();
        st.close();
        rs.close();

    }

}
```

Aufgabe 4 a)

```
public class Main {
    public static void main(String[] args) throws SQLException {
        String adresse =
"jdbc:postgresql://localhost:5432/postgres?currentSchema=%s";
        String schema = "chinook";
        String url = String.format(adresse, schema);
        Properties props = new Properties();
        props.setProperty("user", "postgres");
        props.setProperty("password", "postgres");

        // Verbindungsaufbau
        Connection conn = DriverManager.getConnection(url,
props);

        // SQL statement vorbereiten
        Statement st = conn.createStatement();
        String stcommand = "SELECT * FROM employee WHERE
last_name = ";

        // Eingabe simulieren
        String lastname = "'King'";

        // SQL Befehl ausführen
        ResultSet rs=st.executeQuery(stcommand+lastname);

        // Auswertung

        while(rs.next()){
            String vorname=rs.getString("first_name");
            System.out.println(vorname);
        }

        // Freigabe der Ressourcen
        conn.close();
        st.close();
        rs.close();

    }
}
```

Aufgabe 4 b)

```
public class Main {
    public static void main(String[] args) throws SQLException {
        String adresse =
"jdbc:postgresql://localhost:5432/postgres?currentSchema=%s";
        String schema = "chinook";
        String url = String.format(adresse, schema);
        Properties props = new Properties();
        props.setProperty("user", "postgres");
        props.setProperty("password", "postgres");

        // Verbindungsaufbau
        Connection conn = DriverManager.getConnection(url,
props);

        // SQL statement vorbereiten
        Statement st = conn.createStatement();
        String stcommand = "SELECT * FROM employee WHERE
last_name = ";

        // Eingabe simulieren: zeigen dass mit bestimmten eingaben alles
angezeigt wird
        String lastname = "'grün' OR 1=1;";

        // SQL Befehl ausführen
        ResultSet rs=st.executeQuery(stcommand+lastname);

        // Auswertung

        while(rs.next()){
            String vorname=rs.getString("first_name");
            System.out.println(vorname);
        }

        // Freigabe der Ressourcen
        conn.close();
        st.close();
        rs.close();
    }
}
```

```
    }  
  
}
```

Aufgabe 4 c)

```
public class Main {  
    public static void main(String[] args) throws SQLException {  
        String adresse =  
"jdbc:postgresql://localhost:5432/postgres?currentSchema=%s";  
        String schema = "chinook";  
        String url = String.format(adresse, schema);  
        Properties props = new Properties();  
        props.setProperty("user", "postgres");  
        props.setProperty("password", "postgres");  
  
        // Verbindungsaufbau  
        Connection conn = DriverManager.getConnection(url,  
props);  
  
        // SQL statement vorbereiten  
        PreparedStatement pst=conn.prepareStatement("SELECT *  
FROM employee WHERE last_name = ?");  
        // Eingabe simulieren  
        String lastname = "King";  
  
        pst.setString(1,lastname);  
  
  
        // SQL Befehl ausführen  
        ResultSet rs=pst.executeQuery();  
  
  
        // Auswertung -  
  
        while(rs.next()){  
            String vorname=rs.getString("first_name");  
            System.out.println(vorname);  
        }  
  
  
        // Freigabe der Ressourcen  
        conn.close();  
        pst.close();  
    }  
}
```

```
        rs.close();  
    }  
}
```

Test, ob SQL-Injection verhindert wird:
➔ `String lastname = "'grün' OR 1=1;"`;

Aufgabe 5 a)

```
public class Main {
    public static void main(String[] args) throws SQLException {
        String adresse =
"jdbc:postgresql://localhost:5432/postgres?currentSchema=%s";
        String schema = "chinook";
        String url = String.format(adresse, schema);
        Properties props = new Properties();
        props.setProperty("user", "postgres");
        props.setProperty("password", "postgres");

        // Verbindungsaufbau
        Connection conn = DriverManager.getConnection(url,
props);

        // SQL statement vorbereiten
        Statement st = conn.createStatement();
        // a) Mitarbeiter mit Nummer 2 auf inactive umbenennen
        String stcommand = "UPDATE employee SET last_name =
'inctive' WHERE employee_id = 2;";
        st.executeUpdate(stcommand);

        // Tabelleninhalt ausgeben um Ergebnis anzuzeigen
        // SQL statement vorbereiten
        stcommand = "select * from employee;";
        // SQL Befehl ausführen
        ResultSet rs = st.executeQuery(stcommand);
        // Auswertung - Ausgabe der Tabelle employee
        while (rs.next()) {
            Integer ID = rs.getInt("employee_id");
            String NName = rs.getString("last_name");
            String VName = rs.getString("first_name");
            System.out.print("      ");
            System.out.print(ID);
            System.out.print("      ");
            System.out.println(NName);
            System.out.println(VName);
        }

        // Freigabe der Ressourcen
        conn.close();
        st.close();
        rs.close();
    }
}
```

```

    }

}

```

Aufgabe 5 b)

```

public class Main {
    public static void main(String[] args) throws SQLException {
        String adresse =
"jdbc:postgresql://localhost:5432/postgres?currentSchema=%s";
        String schema = "chinook";
        String url = String.format(adresse, schema);
        Properties props = new Properties();
        props.setProperty("user", "postgres");
        props.setProperty("password", "postgres");

        // Verbindungsaufbau
        Connection conn = DriverManager.getConnection(url, props);

        // SQL statement vorbereiten
        Statement st = conn.createStatement();
        // b) neuer Mitarbeiter anlegen
        String stcommand = "INSERT INTO employee (employee_id,
last_name, first_name) VALUES (192, 'Mayer', 'Patrick');";
        st.executeUpdate(stcommand);

        // Tabelleninhalt ausgeben um Ergebnis anzuzeigen
        // SQL statement vorbereiten
        stcommand = "select * from employee;";
        // SQL Befehl ausführen
        ResultSet rs = st.executeQuery(stcommand);
        // Auswertung - Ausgabe der Tabelle employee
        while (rs.next()) {
            Integer ID = rs.getInt("employee_id");
            String NName = rs.getString("last_name");
            String VName = rs.getString("first_name");
            System.out.print("      ");
            System.out.print(ID);
            System.out.print("      ");
            System.out.println(NName);
            System.out.println(VName);
        }

        // Freigabe der Ressourcen
        conn.close();
        st.close();
        rs.close();

    }
}

```

```
}
```

Aufgabe 5 c)

```
public class Main {
    public static void main(String[] args) throws SQLException {
        String adresse =
"jdbc:postgresql://localhost:5432/postgres?currentSchema=%s";
        String schema = "chinook";
        String url = String.format(adresse, schema);
        Properties props = new Properties();
        props.setProperty("user", "postgres");
        props.setProperty("password", "postgres");

        // Verbindungsaufbau
        Connection conn = DriverManager.getConnection(url, props);

        // SQL statement vorbereiten
        Statement st = conn.createStatement();

        // c) Tabelleninhalt ausgeben um Ergebnis anzuzeigen
        // SQL statement vorbereiten
        String stcommand = "select * from employee;";
        // SQL Befehl ausführen
        ResultSet rs = st.executeQuery(stcommand);
        // Auswertung - Ausgabe der Tabelle employee
        while (rs.next()) {
            Integer ID = rs.getInt("employee_id");
            String NName = rs.getString("last_name");
            String VName = rs.getString("first_name");
            System.out.print("      ");
            System.out.print(ID);
            System.out.print("      ");
            System.out.println(NName);
            System.out.println(VName);
        }

        // Freigabe der Ressourcen
        conn.close();
        st.close();
        rs.close();

    }
}
```